Explorador de Labirintos: Comparação entre Busca em Largura e Busca em Profundidade em Grafos

1º Matheus Ulrich Pontes
Engenharia da Computação
Fundação Herminio Ometto
Araras, Brasil
RA 113367

2º Mauricio Guilherme
Engenharia da Computação
Fundação Herminio Ometto
Araras, Brasil
RA 113586

3º Pedro Oliveira
Engenharia da Computação
Fundação Herminio Ometto
Araras, Brasil
RA 113364

4º Gustavo de Paula Oliveira
Engenharia da Computação
Fundação Herminio Ometto
Araras, Brasil
RA 114044

***Resumo*—Este trabalho apresenta um software que modela um labirinto como um grafo e executa, sobre ele, os algoritmos de busca em largura (BFS) e busca em profundidade (DFS), exibindo a exploração de forma visual e comparando seus resultados. Cada posição livre do labirinto é um vértice, cada passagem entre posições vizinhas é uma aresta, e o grafo é armazenado em lista de adjacência. O sistema, escrito em Python com interface em Tkinter, gera labirintos em três tamanhos, anima as posições visitadas e o caminho encontrado e reporta a quantidade de posições visitadas, o comprimento do caminho e o tempo de execução. Em um experimento com cem labirintos por configuração, a BFS sempre encontrou o caminho de menor comprimento, enquanto o caminho do DFS foi, em média, de 1,49 a 2,46 vezes mais longo, dependendo do tamanho. Quando o labirinto não possui ciclos, os dois algoritmos encontraram caminhos idênticos, o que confirma a relação entre ciclos e diversidade de rotas. Em contrapartida, o DFS visitou menos posições na maioria dos casos, o que mostra que a garantia de caminho mínimo tem um custo de exploração.**

***Palavras-chave*—teoria dos grafos, busca em largura, busca em profundidade, labirinto, lista de adjacência, visualização de algoritmos**

I. Introdução

Grafos são uma linguagem para descrever conexões, dependências e caminhos, e aparecem em problemas tão distintos quanto rotas, redes sociais e redes de computadores [1]. Um labirinto é um exemplo clássico e intuitivo desse tipo de problema: encontrar a saída equivale a encontrar um caminho entre dois vértices de um grafo. O problema do menor caminho em um labirinto foi tratado por Moore já em 1959 [3], e a busca em profundidade ocupa lugar central na análise de grafos desde os trabalhos de Tarjan [4].

O objetivo deste trabalho é demonstrar, de forma prática e visual, como dois algoritmos tradicionais de busca em grafos, a busca em largura (*Breadth-First Search*, BFS) e a busca em profundidade (*Depth-First Search*, DFS), percorrem um mesmo labirinto e como seus comportamentos diferem. Para isso, desenvolvemos um software que:

- transforma um labirinto em um grafo $G = (V, E)$ armazenado em lista de adjacência;
- implementa BFS e DFS sem bibliotecas de grafos prontas;
- anima a exploração e destaca o caminho encontrado;
- compara os algoritmos por posições visitadas, comprimento do caminho e tempo.

Além do software, conduzimos um experimento reprodutível com labirintos gerados automaticamente, que permite relacionar os resultados à teoria vista em aula, em particular às noções de caminho, distância, ciclo e árvore.

O restante do artigo está organizado da seguinte forma. A Seção II resume a fundamentação teórica. A Seção III descreve a modelagem do labirinto como grafo. A Seção IV apresenta a implementação. A Seção V detalha a metodologia experimental, e a Seção VI traz os resultados e sua discussão. As limitações e a conclusão encerram o texto.

II. Fundamentação Teórica

A. Grafos e sua representação

Um grafo é definido como $G = (V, E)$, em que V é o conjunto de vértices e E o conjunto de arestas [1]. Um grafo é *simples* quando não possui laços nem arestas múltiplas e *não dirigido* quando suas arestas não têm sentido. O *grau* de um vértice é o número de arestas que incidem nele. Quando as arestas não têm peso, o grafo é dito não ponderado.

Duas representações computacionais são básicas: a lista de adjacência, em que cada vértice guarda a lista de seus vizinhos, e a matriz de adjacência, que indica a existência de aresta entre cada par de vértices. A lista ocupa memória proporcional a $|V| + |E|$ e é indicada para percorrer vizinhos, enquanto a matriz ocupa memória proporcional a $|V|^2$ e favorece o teste de existência de uma aresta [1].

B. Caminhos, ciclos e conectividade

Um *passeio* é uma sequência de vértices e arestas que pode repetir vértices e arestas; uma *trilha* não repete arestas; e um *caminho* não repete vértices nem arestas. Todo caminho é uma trilha e toda trilha é um passeio, nunca o contrário [1]. O *comprimento* de um caminho é o número de arestas percorridas, e a *distância* entre dois vértices é o comprimento do menor caminho entre eles. Um *ciclo* é um caminho fechado, e um grafo sem ciclos é dito acíclico.

Um grafo é *conexo* quando existe caminho entre qualquer par de vértices; caso contrário, divide-se em *componentes*

conexas. Vale para todo grafo que a soma dos graus dos vértices é

$$\sum_{v \in V} \text{grau}(v) = 2\,|E|, \tag{1}$$

e um grafo conexo e acíclico com n vértices, chamado árvore, tem exatamente $n - 1$ arestas [1]. Em um grafo conexo, cada aresta além das $n - 1$ de uma árvore cria um ciclo independente. Essa última observação é um complemento ao material da disciplina (o número ciclomático [2]), e a usamos para quantificar os ciclos dos labirintos.

C. BFS e DFS (conteúdo complementar)

Os algoritmos BFS e DFS são anunciados no material da disciplina como assunto da aula seguinte à de caminhos e conectividade. Por isso, sua descrição aqui segue a formulação tradicional da literatura [2], e não o material da disciplina.

A **BFS** usa uma fila. Partindo do vértice inicial, visita todos os vértices à distância 1, depois os de distância 2, e assim sucessivamente, avançando por níveis. Em um grafo não ponderado, o primeiro caminho que chega ao destino tem o menor número possível de arestas, isto é, seu comprimento é a distância definida na seção anterior. Esse é o princípio do algoritmo de Moore para labirintos [3].

O **DFS** usa uma pilha (ou recursão). Segue um caminho até onde for possível e, ao encontrar um vértice sem vizinhos novos, retorna ao último ponto com alternativas. Ele encontra um caminho, caso exista, mas não garante que seja o menor [2], [4].

Com a lista de adjacência, ambos têm custo de tempo $O(|V|+|E|)$ [2], coerente com o custo da representação visto na Seção II-A.

III. Modelagem do Labirinto como Grafo

O labirinto é uma grade em que cada posição é livre ou parede. A Tabela I resume a correspondência com a teoria.

Tabela I
Correspondência entre o labirinto e a teoria dos grafos

Labirinto	**Conceito**	**Aula**
Posição livre	Vértice $v \in V$	01
Parede	Posição bloqueada (não é vértice)	—
Passagem entre posições livres vizinhas	Aresta $\{u, v\} \in E$	01
Estrutura de dados	Lista de adjacência	02
Trajeto da entrada à saída	Caminho (sem repetir vértices)	03
Número de passos	Comprimento (nº de arestas)	03
Menor trajeto	Distância	03
Saída inalcançável	Entrada e saída em componentes conexas distintas	03

Os movimentos permitidos são para cima, para a direita, para baixo e para a esquerda. Duas posições livres vizinhas nessas quatro direções são ligadas por uma aresta. O grafo resultante é simples, não dirigido e não ponderado: se é possível ir de uma posição a outra, também é possível voltar, e todo passo tem o mesmo custo. Em consequência, o comprimento de um caminho é simplesmente o seu número de passos.

Como cada posição tem no máximo quatro vizinhos, o grau de todo vértice é no máximo quatro e o grafo é esparso. Segundo os custos da Seção II-A, a lista de adjacência é a representação adequada: a matriz de adjacência ocuparia memória proporcional a $|V|^2$, enquanto a lista ocupa memória proporcional a $|V| + |E|$. Um detalhe importante é que, nesta modelagem, os corredores entre as células do gerador também são vértices, pois cada posição livre da grade é um vértice.

IV. Implementação

O sistema foi escrito em Python 3 com interface em Tkinter, usando apenas a biblioteca padrão, sem bibliotecas de grafos, para que a estrutura e os algoritmos sejam integralmente implementados pelo grupo. O código é dividido em quatro módulos com responsabilidades distintas:

- `grafo.py`: classe de grafo com lista de adjacência e a conversão da grade em $G = (V, E)$;
- `labirinto.py`: geração dos labirintos;
- `algoritmos.py`: BFS, DFS e a verificação de caminhos;
- `interface.py`: interface gráfica e animação.

Os módulos de grafo e algoritmos não conhecem a interface. Cada algoritmo devolve a ordem em que visitou os vértices, o caminho encontrado e o tempo gasto, e a interface apenas reproduz essas listas na tela. Assim, os mesmos dados alimentam a animação e a comparação.

A. Geração dos labirintos

O gerador cria, primeiro, um labirinto perfeito por *backtracking* iterativo com pilha. Esse procedimento produz um grafo conexo e acíclico, ou seja, uma árvore com $n - 1$ arestas, em que existe um único caminho entre quaisquer duas posições. Em seguida, remove-se um número adicional de paredes, igual a uma fração do número de células, o que cria ciclos e, portanto, rotas alternativas. A entrada fica no canto superior esquerdo e a saída, no inferior direito. Como o labirinto é conexo por construção, sempre há solução. Há três tamanhos: Pequeno, Médio e Grande, com 10×10, 20×20 e 30×30 células (grades de 21×21, 41×41 e 61×61 posições).

B. Algoritmos

Na BFS, o vértice é marcado ao ser enfileirado, de modo que nenhum vértice entra duas vezes na fila. No DFS, implementado de forma iterativa, o vértice é marcado ao ser desempilhado e os vizinhos são empilhados em ordem inversa; como a pilha retira primeiro o último elemento, a exploração segue a ordem fixa cima, direita, baixo e esquerda, exatamente como faria a versão recursiva. Essa ordem fixa torna os resultados do DFS determinísticos e reproduzíveis.

Em ambos, guarda-se o vértice de origem de cada descoberta (o "pai"). O caminho é reconstruído partindo da saída e voltando pelos pais até a entrada. Como cada vértice é descoberto uma única vez, o resultado não repete vértices e é, portanto, um caminho no sentido da teoria, e não apenas um passeio.

C. Interface

A interface (Fig. 1) apresenta o labirinto em um *Canvas*, com a entrada em verde e a saída em vermelho. O usuário escolhe o tamanho, gera um novo labirinto, executa BFS ou DFS e ajusta a velocidade da animação por um controle deslizante. As posições visitadas são coloridas (azul para a BFS e laranja para o DFS) e o caminho encontrado é destacado. Um painel mostra posições visitadas, comprimento e tempo de cada algoritmo, além de uma comparação textual. Há também informações do grafo gerado: $|V|$, $|E|$ e o número de arestas além de $|V| - 1$.

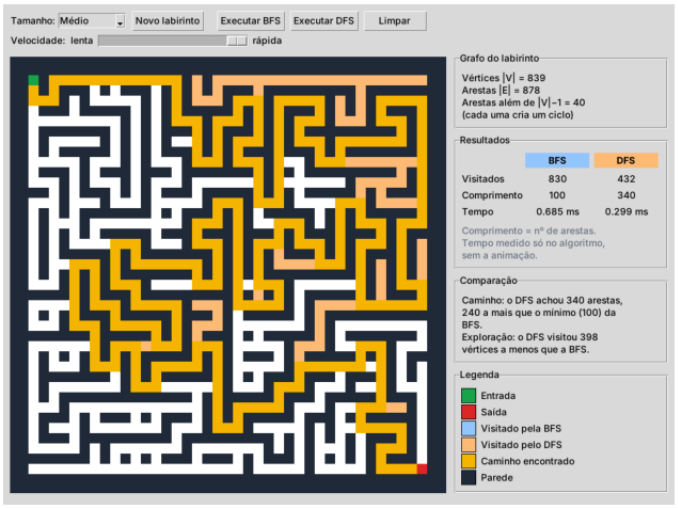


Figura 1. Interface do sistema após a execução de BFS e DFS em um labirinto de tamanho Médio. O caminho do DFS está destacado; a tabela guarda os resultados dos dois algoritmos.

V. Metodologia Experimental

Experimento 1 (tamanho). Para cada tamanho (Pequeno, Médio e Grande), geramos 100 labirintos com sementes fixas de 0 a 99, com remoção de paredes extras equivalente a 10% do número de células. Em cada labirinto, executamos BFS e DFS entre a entrada e a saída.

Experimento 2 (ciclos). No tamanho Médio, repetimos o procedimento com 0%, 2%, 5%, 10% e 20% de paredes extras, também com 100 labirintos por configuração, para observar o efeito dos ciclos.

Métricas. Registramos a quantidade de posições visitadas, o comprimento do caminho (número de arestas) e o tempo de execução do algoritmo, sem a animação. Para reduzir ruído, o tempo de cada algoritmo em cada labirinto é o mínimo de cinco execuções, e a tabela apresenta a mediana entre os labirintos. Os tempos dependem do computador e da implementação em Python e, por isso, são tratados como informação complementar.

Validação. Em todas as execuções, verificamos automaticamente que os caminhos encontrados são válidos (não repetem vértices e cada par consecutivo é uma aresta) e que o comprimento da BFS nunca supera o do DFS. Em três labirintos pequenos de teste com solução, confirmamos ainda, por enumeração exaustiva de todos os caminhos, que o comprimento obtido pela BFS é o mínimo. Em um quarto labirinto de teste, cuja saída é inacessível a partir da entrada, confirmamos que BFS e DFS concordam que não existe caminho. Verificamos também a Equação (1) e a simetria das arestas nos grafos gerados.

VI. Resultados e Discussão

A Fig. 2 mostra BFS e DFS no mesmo labirinto Pequeno. Esse labirinto (semente 67) não foi escolhido por ser o mais favorável, e sim por ter a razão entre os comprimentos do DFS e da BFS mais próxima da média do experimento. A BFS encontrou um caminho de 48 arestas visitando 168 posições, e o DFS encontrou um caminho de 72 arestas visitando 77 posições.

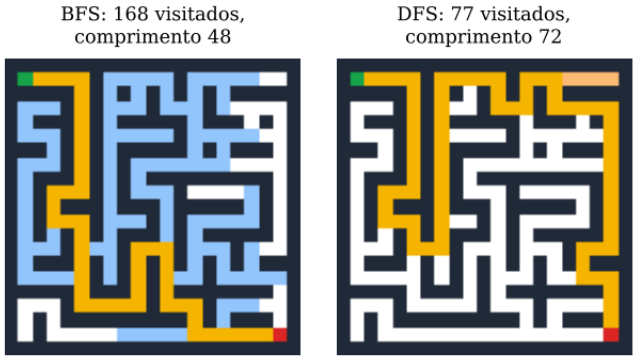


Figura 2. BFS e DFS no mesmo labirinto Pequeno. Verde: entrada; vermelho: saída; azul: visitado pela BFS; laranja: visitado pelo DFS; amarelo: caminho encontrado.

A. Efeito do tamanho

A Tabela II resume o Experimento 1. Nos três tamanhos, o comprimento do caminho do DFS foi maior que o da BFS: em média, 1,49 vez no tamanho Pequeno, 1,98 vez no Médio e 2,46 vezes no Grande. A proporção de labirintos em que o DFS encontrou um caminho estritamente mais longo foi de 86%, 99% e 100%, respectivamente. A BFS nunca foi pior que o DFS em comprimento, em nenhum dos 300 labirintos, como garante a teoria.

Em contrapartida, o DFS visitou menos posições que a BFS em 96%, 99% e 100% dos labirintos. Em média, a BFS visitou cerca de 84%, 93% e 95% dos vértices nos três tamanhos, contra cerca de 43%, 35% e 35% do DFS. Uma explicação plausível é que a saída fica no canto oposto à entrada, o ponto mais distante: como a BFS explora por níveis de distância, precisa visitar quase todas as posições mais próximas da entrada antes de alcançar a saída, ao passo que o DFS não tem essa restrição. Essa explicação é uma interpretação nossa e não foi testada isoladamente. O desvio padrão do número de posições visitadas pelo DFS é grande em comparação ao da BFS, o que indica que seu custo de exploração depende muito do labirinto específico.

Os tempos medidos foram da ordem de centésimos a décimos de milissegundo e acompanham o número de posições

Tabela II
RESULTADOS POR TAMANHO (MÉDIA ± DESVIO PADRÃO DE 100 LABIRINTOS; TEMPO: MEDIANA)

Tamanho	$\|V\|$	Posições visitadas		Comprimento (arestas)		Razão	Tempo (ms)	
		BFS	DFS	BFS	DFS	DFS/BFS	BFS	DFS
Pequeno	209	175,5 ± 31,2	89,7 ± 36,4	46,5 ± 7,8	69,2 ± 21,6	1,49	0,05	0,03
Médio	839	776,7 ± 73,9	290,0 ± 141,6	104,7 ± 16,4	204,0 ± 73,4	1,98	0,22	0,10
Grande	1889	1792,4 ± 151,8	657,0 ± 396,6	161,3 ± 22,8	391,6 ± 150,2	2,46	0,52	0,23

visitadas, de modo que o DFS foi mais rápido neste cenário. Dada a escala dos valores e sua dependência do ambiente, não os usamos como evidência principal.

B. Efeito dos ciclos

A Tabela III e a Fig. 3 resumem o Experimento 2. Sem paredes extras, o grafo é uma árvore: existe um único caminho entre entrada e saída e, portanto, BFS e DFS encontraram caminhos idênticos nos 100 labirintos (razão 1,00 e 0% de casos em que o DFS foi mais longo). À medida que ciclos são adicionados, a BFS aproveita os atalhos e o comprimento do seu caminho diminui, de 307,5 para 85,2 arestas, enquanto o DFS continua seguindo rotas longas, e a razão entre os comprimentos sobe de 1,00 até 2,16. Esse resultado liga o comportamento dos algoritmos à propriedade de árvores vista em aula: o comprimento do caminho só tem diversidade quando há ciclos.

Tabela III
EFEITO DOS CICLOS NO TAMANHO MÉDIO (100 LABIRINTOS POR LINHA). AS COLUNAS BFS E DFS MOSTRAM O COMPRIMENTO MÉDIO DO CAMINHO, EM ARESTAS; A ÚLTIMA COLUNA, A PROPORÇÃO DE LABIRINTOS EM QUE O CAMINHO DO DFS FOI MAIS LONGO

Paredes extras	Ciclos	BFS	DFS	DFS/BFS	DFS maior
0%	0	307,5	307,5	1,00	0%
2%	8	191,4	265,0	1,44	81%
5%	20	141,7	224,2	1,63	96%
10%	40	104,7	204,0	1,98	99%
20%	80	85,2	184,4	2,16	100%

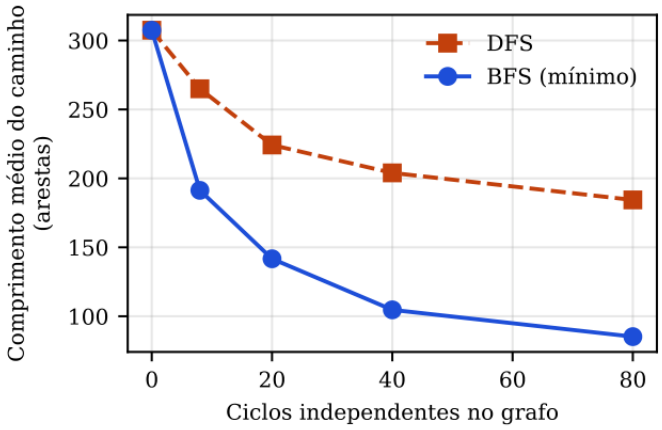


Figura 3. Comprimento médio do caminho em função do número de ciclos independentes (tamanho Médio).

C. Limitações

Os resultados valem para as condições do experimento e não devem ser generalizados como propriedades gerais dos algoritmos. Todos os labirintos vêm de um único gerador, com entrada e saída sempre em cantos opostos. O DFS depende da ordem dos vizinhos, aqui fixa em cima, direita, baixo e esquerda; outra ordem alteraria os números do DFS, mas não o comprimento do caminho da BFS. O modelo considera apenas quatro direções e não possui pesos. Por fim, os tempos foram medidos em Python e variam com o ambiente.

VII. CONCLUSÃO

Neste trabalho, modelamos um labirinto como um grafo simples, não dirigido e não ponderado, em lista de adjacência, e implementamos BFS e DFS com visualização passo a passo. O experimento com 100 labirintos por configuração mostrou que a BFS sempre encontra um caminho de menor comprimento, enquanto o caminho do DFS é, em média, de 1,49 a 2,46 vezes mais longo, razão que cresce com o tamanho do labirinto. Quando o labirinto é uma árvore, os dois algoritmos coincidem, e a diferença surge com a presença de ciclos. O DFS, por outro lado, visitou menos posições na grande maioria dos casos, de modo que a garantia de caminho mínimo da BFS tem custo de exploração neste cenário.

Como trabalhos futuros, propomos variar a posição da entrada e da saída, usar ordem aleatória dos vizinhos no DFS, permitir movimentos em oito direções, exibir as duas explorações lado a lado e estender o modelo a grafos ponderados, com algoritmos de caminho mínimo como Dijkstra.

REFERÊNCIAS

[1] T. Milani, "Teoria dos Grafos: Aula 01 (Introdução, conceitos básicos e aplicações), Aula 02 (Representação de grafos) e Aula 03 (Caminhos, ciclos e conectividade)," material didático, Engenharia da Computação, 10º período.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA: MIT Press, 2022.

[3] E. F. Moore, "The shortest path through a maze," in *Proc. Int. Symp. Theory of Switching*, Cambridge, MA: Harvard Univ. Press, 1959, pp. 285–292.

[4] R. Tarjan, "Depth-first search and linear graph algorithms," *SIAM J. Comput.*, vol. 1, no. 2, pp. 146–160, 1972.